Code Structure and Multiple Files

Let's stop for a second to think about how to structure the code, particularly in **large projects** with multiple files.

Circular Imports

The class `Hero` has a reference to the class `Team` internally.

But the class `Team` also has a reference to the class `Hero`.

So, if those two classes were in separate files and you tried to import the classes in each other's file directly, it would result in a **circular import**. 🔄

And Python will not be able to handle it and will throw an error. 🚨

But we actually want to *mean* that **circular reference**, because in our code, we would be able to do crazy things like:

```
hero.team.heroes[0].team.heroes[1].team.heroes[2].name
```

And that circular reference is what we are expressing with these **relationship attributes**, that:

- A hero can have a team
  - That team can have a list of heroes
    - Each of those heroes can have a team
      - ...and so on.

Let's see different strategies to **structure the code** accounting for this.

Single Module for Models

This is the simplest way. ✨

In this solution we are still using **multiple files**, for the `models`, for the `database`, and for the `app`.

And we could have any **other files** necessary.

But in this first case, all the models would live in a **single file**.

The file structure of the project could be:

```
.
├── project
    ├── __init__.py
    ├── app.py
    ├── database.py
    └── models.py
```

We have 3 **Python modules** (or files):

- `app`

- `database`
- `models`

And we also have an empty `__init__.py` file to make this project a "**Python package**" (a collection of Python modules). This way we can use **relative imports** in the `app.py` file/module, like:

```
from .models import Hero, Team
from .database import engine
```

We can use these relative imports because, for example, in the file `app.py` (the `app` module) Python knows that it is **part of our Python package** because it is in the same directory as the file `__init__.py`. And all the Python files on the same directory are part of the same Python package too.

Models File

You could put all the database Models in a single Python module (a single Python file), for example `models.py`:

Python 3.10+

```
from sqlmodel import Field, Relationship, SQLModel


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")
```

This way, you wouldn't have to deal with circular imports for other models.

And then you could import the models from this file/module in any other file/module in your application.

Database File

Then you could put the code creating the **engine** and the function to create all the tables (if you are not using migrations) in another file `database.py`:

Python 3.10+

```
from sqlmodel import SQLModel, create_engine

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
```

This file would also be imported by your application code, to use the shared **engine** and to get and call the function `create_db_and_tables()`.

Application File

Finally, you could put the code to create the **app** in another file `app.py`:

Python 3.10+

```
from sqlmodel import Session

from .database import create_db_and_tables, engine
from .models import Hero, Team


def create_heroes():
    with Session(engine) as session:
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        session.add(hero_deadpond)
        session.commit()

        session.refresh(hero_deadpond)

        print("Created hero:", hero_deadpond)
        print("Hero's team:", hero_deadpond.team)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Here we import the models, the engine, and the function to create all the tables and then we can use them all internally.

Order Matters

Remember that Order Matters when calling `SQLModel.metadata.create_all()`?

The point of that section in the docs is that you have to import the module that has the models **before** calling `SQLModel.metadata.create_all()`.

We are doing that here, we import the models in `app.py` and **after** that we create the database and tables, so we are fine and everything works correctly. 👌

Run It in the Command Line

Because now this is a larger project with a **Python package** and not a single Python file, we **cannot** call it just passing a single file name as we did before with:

```
$ python app.py
```

Now we have to tell Python that we want it to execute a *module* that is part of a package:

```
$ python -m project.app
```

The `-m` is to tell Python to call a *module*. And the next thing we pass is a string with `project.app`, that is the same format we would use in an **import**:

```
import project.app
```

Then Python will execute that module *inside* of that package, and because Python is executing it directly, the same trick with the **main block** that we have in `app.py` will still work:

```
if __name__ == '__main__':
    main()
```

So, the output would be:

```bash
```

Make Circular Imports Work

Let's say that for some reason you hate the idea of having all the database models together in a single file, and you really want to have **separate files**: a `hero_model.py` file and a `team_model.py` file.

You can also do it. 😎 There's a couple of things to keep in mind. 🤓

> **Warning**
>
> This is a bit more advanced.
>
> If the solution above already worked for you, that might be enough for you, and you can continue in the next chapter. 🤓

Let's assume that now the file structure is:

```
.
├── project
    ├── __init__.py
    ├── app.py
    ├── database.py
    ├── hero_model.py
    └── team_model.py
```

Circular Imports and Type Annotations

The problem with circular imports is that Python can't resolve them at *runtime*.

But when using Python **type annotations** it's very common to need to declare the type of some variables with classes imported from other files.

And the files with those classes might **also need to import** more things from the first files.

And this ends up *requiring* the same **circular imports** that are not supported in Python at *runtime*.

Type Annotations and Runtime

But these **type annotations** we want to declare are not needed at *runtime*.

In fact, remember that we used `list["Hero"]`, with a `"Hero"` in a string?

For Python, at runtime, that is **just a string**.

So, if we could add the type annotations we need using the **string versions**, Python wouldn't have a problem.

But if we just put strings in the type annotations, without importing anything, the editor wouldn't know what we mean, and wouldn't be able to help us with **autocompletion** and **inline errors**.

So, if there was a way to "import" some things that act as "imported" only while editing the code but not at *runtime*, that would solve it... And it exists! Exactly that. 🎉

Import Only While Editing with `TYPE_CHECKING`

To solve it, there's a special trick with a special variable `TYPE_CHECKING` in the `typing` module.

It has a value of `True` for editors and tools that analyze the code with the type annotations.

But when Python is executing, its value is `False`.

So, we can use it in an `if` block and import things inside the `if` block. And they will be "imported" only for editors, but not at runtime.

Hero Model File

Using that trick of `TYPE_CHECKING` we can "import" the `Team` in `hero_model.py`:

Python 3.10+

```python
from typing import TYPE_CHECKING, Optional

from sqlmodel import Field, Relationship, SQLModel

if TYPE_CHECKING:
    from .team_model import Team


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Optional["Team"] = Relationship(back_populates="heroes")
```

Have in mind that now we *have* to put the annotation of `Team` as a string: `"Team"`, so that Python doesn't have errors at runtime.

Team Model File

We use the same trick in the `team_model.py` file:

Python 3.10+

```python
from typing import TYPE_CHECKING

from sqlmodel import Field, Relationship, SQLModel

if TYPE_CHECKING:
    from .hero_model import Hero


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")
```

Now we get editor support, autocompletion, inline errors, and **SQLModel** keeps working. 🎉

App File

Now, just for completeness, the `app.py` file would import the models from both modules:

Python 3.10+

```python
from sqlmodel import Session

from .database import create_db_and_tables, engine
from .hero_model import Hero
from .team_model import Team


def create_heroes():
    with Session(engine) as session:
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        session.add(hero_deadpond)
        session.commit()

        session.refresh(hero_deadpond)

        print("Created hero:", hero_deadpond)
        print("Hero's team:", hero_deadpond.team)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

And of course, all the tricks with `TYPE_CHECKING` and type annotations in strings are **only needed in the files with circular imports**.

As there are no circular imports with `app.py`, we can just use normal imports and use the classes as normally here.

And running that achieves the same result as before:

```bash
```

Recap

For the **simplest cases** (for most of the cases) you can just keep all the models in a single file, and structure the rest of the application (including setting up the **engine**) in as many files as you want.

And for the **complex cases** that really need separating all the models in different files, you can use the `TYPE_CHECKING` to make it all work and still have the best developer experience with the best editor support. ✨